1. Printing on Screen

Q1) Introduction to the print() function in Python.

The **print()** function is used to **display output on the screen** in Python.

Example:

```
print("Hello World")
```

Output:

```
Hello World
```

Q2) Formatting outputs using f-strings and format().

1. Using f-strings (Recommended Method)

f-strings allow you to insert variables directly inside a string using {}.

```
name = "Rana"
age = 22

print(f"My name is {name} and I am {age} years old.")
```

Output:

```
My name is Rana and I am 22 years old.
```

Formatting Numbers

```
price = 99.4567
print(f"Price: {price:.2f}")
```

Output:

```
Price: 99.46
```

.2f → shows 2 decimal places

2. Using format() Method

The format() method inserts values into {} placeholders.

```
name = "Rana"
age = 22

print("My name is {} and I am {} years old.".format(name, age))
```

Output:

```
My name is Rana and I am 22 years old.
```

Formatting Numbers

```
price = 99.4567
print("Price: {:.2f}".format(price))
```

Output:

```
Price: 99.46
```

2. Reading Data from Keyboard

Q1) Using the input() function to read user input from the keyboard.

The input() function is used to take user input from the keyboard.

It pauses the program, waits for the user to type something, and returns the entered value.

Basic Syntax

```
name = input("Enter your name: ")
print("Hello", name)
```

Example Output:

```
Enter your name: Rana
Hello Rana
```

Q2) Converting user input into different data types (e.g., int, float, etc.).

The input() function always returns data as a **string**.
To use it as a number or another type, we convert it using type casting.

Converting to Integer (int)

```
age = int(input("Enter your age: "))
print("Age after 5 years:", age + 5)
```

int() converts the input into a whole number.

Converting to Float (float)

```
price = float(input("Enter price: "))
print("Price with tax:", price * 1.18)
```

float() converts the input into a decimal number.

Converting to String (str)

```
num = 100
text = str(num)
print("Number is " + text)
```

str() converts data into string format.

Converting to Boolean (bool)

```
value = bool(input("Enter something: "))
print(value)
```

If the user enters anything, it returns True.
If empty, it returns False.

3. Opening and Closing Files

Q1) Opening files in different modes ('r', 'w', 'a', 'r+', 'w+').

In Python, we use the open() function to open a file.

Syntax

```
file = open("filename.txt", "mode")
```

File Modes

1. 'r' – Read Mode

- Opens the file for reading.
- Gives an error if the file does not exist.
- Default mode.

```
file = open("data.txt", "r")
```

2. 'w' – Write Mode

- Opens the file for writing.
- Creates the file if it does not exist.
- Deletes old content if the file already exists.

```
file = open("data.txt", "w")
```

3. 'a' – Append Mode

- Opens the file for writing.
- Creates the file if it does not exist.
- Adds new content at the end without deleting old content.

```
file = open("data.txt", "a")
```

4. 'r+' – Read and Write Mode

- Opens the file for both reading and writing.
- File must exist.
- Does not delete old content.

```
file = open("data.txt", "r+")
```

5. 'w+' – Write and Read Mode

- Opens the file for both writing and reading.
- Creates the file if it does not exist.
- Deletes old content if the file exists.

```
file = open("data.txt", "w+")
```

Important

Always close the file after use:

```
file.close()
```

Or use with (recommended):

```
with open("data.txt", "r") as file:
    content = file.read()
```

Q2) Using the open() function to create and access files.

Using the open() Function to Create and Access Files

In Python, the open() function is used to create or access a file.

Basic syntax

```
file = open("file.txt", "mode")
```

Creating a file

If the file does not exist, it will be created using write mode.

```
file = open("data.txt", "w")
file.write("Hello World")
file.close()
```

Accessing (reading) a file

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

Recommended way (using with statement)

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

Q3) Closing files using close().

In Python, after opening and using a file, we should close it using the close() method.

Closing a file frees system memory and saves changes properly.

Example:

```
file = open("data.txt", "r")
content = file.read()
```

```
print(content)
file.close()
```

Here, file.close() closes the file after reading.

Important:
If you forget to close a file, it may cause memory problems or data loss.

Better Method (Recommended):

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

When using with, Python automatically closes the file.

4. Reading and Writing Files

Q1) Reading from a file using read(), readline(), readlines().

Reading from a File in Python

In Python, we use read(), readline(), and readlines() to read data from a file.

Using read()
Reads the entire file content at once.

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

Using readline()
Reads only one line at a time.

```
file = open("data.txt", "r")
line = file.readline()
print(line)
file.close()
```

Using readlines()
Reads all lines and returns them as a list.

```
file = open("data.txt", "r")
lines = file.readlines()
print(lines)
file.close()
```

Simple difference:
read() → reads whole file
readline() → reads one line
readlines() → reads all lines as a list

Q2) Writing to a file using write() and writelines().

Writing to a File in Python

In Python, we use write() and writelines() to write data into a file.

Using write()
It writes a single string into the file.

```
file = open("data.txt", "w")
file.write("Hello World\n")
file.write("Welcome to Python")
file.close()
```

write() writes text as it is. If you want a new line, you must add \n.

Using writelines()
It writes multiple strings from a list into the file.

```
file = open("data.txt", "w")
lines = ["Apple\n", "Banana\n", "Mango\n"]
file.writelines(lines)
file.close()
```

writelines() does not automatically add new lines, so you must include \n in each string.

Simple difference:
write() → writes one string
writelines() → writes multiple strings (from a list)

5. Exception Handling

Q1) Introduction to exceptions and how to handle them using try, except, and finally.

An exception is an error that happens during program execution.
If not handled, the program will stop.

To handle errors, Python uses try, except, and finally.

Basic Structure

```
try:
    # code that may cause error
except:
    # code to handle error
finally:
    # code that always runs
```

Example

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print(result)
except ZeroDivisionError:
    print("You cannot divide by zero.")
except ValueError:
    print("Invalid input. Please enter a number.")
finally:
    print("Program finished.")
```

Explanation

try → Write the code that may cause an error.
except → Handles the error if it happens.
finally → Runs always, whether error happens or not.

Q2) Understanding multiple exceptions and custom exceptions.

Multiple Exceptions in Python

In Python, there are three main ways to handle multiple exceptions.

1. Using Multiple except Blocks

We write separate except blocks for different errors.

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print(result)

except ZeroDivisionError:
    print("Cannot divide by zero.")
```

```
except ValueError:
    print("Invalid input.")

except TypeError:
    print("Type error occurred.")
```

Each block handles a specific error.

2. Handling Multiple Exceptions in One Block

We can combine multiple exceptions in one except using brackets.

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print(result)

except (ZeroDivisionError, ValueError, TypeError):
    print("An error occurred.")
```

Here, one block handles all listed errors.

3. Using a General Exception

We can use Exception to catch any error.

```
try:

    x = 10 / 0

except Exception as e:
    print("Error:", e)
```

This catches all types of exceptions.

Simple Summary

Method 1 → Separate except blocks
Method 2 → Multiple exceptions in one block
Method 3 → General Exception to catch all errors

Custom Exception

```
class InvalidPasswordError(Exception):

    pass

try:
    password = input("Enter password: ")

    if len(password) < 6:
        raise InvalidPasswordError("Password must be at least 6 characters long.")

    print("Password accepted.")

except InvalidPasswordError as e:
```

```
    print("Error:", e)
```

Simple idea:
We created our own error (InvalidPasswordError).
If the password is too short, we raise the custom exception and handle it using except.

6. Class and Object (OOP Concepts)

Q1) Understanding the concepts of classes, objects, attributes, and methods in Python.

Class
A class is a blueprint or template for creating objects.
It defines the structure and behavior.

Object
An object is an instance of a class.
It represents a real-world entity.

Attributes
Attributes are variables inside a class.
They store data related to the object.

Methods
Methods are functions inside a class.
They define the behavior of the object.

Example:

```
class Student:

    def __init__(self, name, age):   # constructor
        self.name = name            # attribute
        self.age = age              # attribute

    def display(self):              # method
        print("Name:", self.name)
        print("Age:", self.age)


# Creating object
s1 = Student("Rana", 22)

# Calling method
s1.display()
```

Explanation

Student → class
s1 → object
name and age → attributes
display() → method

Simple idea:
Class → blueprint

Object → real thing created from class
Attributes → data
Methods → actions

Q2) Difference between local and global variables.

Difference Between Local and Global Variables

Local Variable
A local variable is declared inside a function.
It can be used only inside that function.

Example:

```
def show():
    x = 10   # local variable
    print(x)

show()
```

Here, x works only inside the function show().

If you try to use x outside the function, it will give an error.

Global Variable
A global variable is declared outside all functions.
It can be used anywhere in the program.

Example:

```
x = 20   # global variable

def show():
    print(x)

show()
print(x)
```

Here, x can be used both inside and outside the function.

Using global Keyword

If you want to change a global variable inside a function, use global.

```
x = 5

def change():
    global x
    x = 10

change()
print(x)
```

Simple Difference

Local variable → inside function, limited use
Global variable → outside function, usable everywhere

7. Inheritance

Q1) Single, Multilevel, Multiple, Hierarchical, and Hybrid inheritance in Python.

1. Single Inheritance
   One child class inherits from one parent class.

```
class Parent:
    def show(self):
        print("Parent class")

class Child(Parent):
    def display(self):
        print("Child class")

obj = Child()
obj.show()
obj.display()
```

2. Multilevel Inheritance
   A class inherits from a child class (chain structure).

```
class Grandparent:
    def show1(self):
        print("Grandparent")

class Parent(Grandparent):
    def show2(self):
        print("Parent")

class Child(Parent):
    def show3(self):
        print("Child")

obj = Child()
obj.show1()
obj.show2()
obj.show3()
```

3. Multiple Inheritance
   One child class inherits from more than one parent class.

```
class Father:
    def show1(self):
        print("Father")

class Mother:
    def show2(self):
        print("Mother")

class Child(Father, Mother):
    def show3(self):
```

```
        print("Child")

obj = Child()
obj.show1()
obj.show2()
obj.show3()
```

4. Hierarchical Inheritance
   Multiple child classes inherit from one parent class.

```
class Parent:
    def show(self):
        print("Parent")

class Child1(Parent):
    pass

class Child2(Parent):
    pass

obj1 = Child1()
obj2 = Child2()

obj1.show()
obj2.show()
```

5. Hybrid Inheritance
   Combination of two or more types of inheritance.

```
class A:
    def show1(self):
        print("Class A")

class B(A):
    def show2(self):
        print("Class B")

class C(A):
    def show3(self):
        print("Class C")

class D(B, C):
    def show4(self):
        print("Class D")

obj = D()
obj.show1()
obj.show2()
obj.show3()
obj.show4()
```

Simple Summary

Single → One parent, one child
Multilevel → Chain inheritance
Multiple → One child, many parents
Hierarchical → One parent, many children
Hybrid → Combination of different inheritance types

Q2) Using the super() function to access properties of the parent class.

The super() function is used to access methods or properties of the parent class inside the child class.

It helps to reuse parent class code.

Example:

```
class Parent:
    def __init__(self, name):
        self.name = name

    def show(self):
        print("Name:", self.name)


class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name)   # calling parent constructor
        self.age = age

    def display(self):
        super().show()          # calling parent method
        print("Age:", self.age)


obj = Child("Rana", 22)
obj.display()
```

Explanation

super().__init__(name) → Calls parent constructor
super().show() → Calls parent method

Simple idea:
super() is used to access parent class properties and methods inside the child class.

8. Method Overloading and Overriding

Q1) Method overloading: defining multiple methods with the same name but different parameters.

Method overloading means defining multiple methods with the same name but different parameters.

In many languages, this works directly.
But in Python, true method overloading is not supported.

If you define multiple methods with the same name, the last one will overwrite the previous one.

Example (Not Working):

```
class Test:
    def add(self, a, b):
        return a + b

    def add(self, a, b, c):
        return a + b + c

obj = Test()
print(obj.add(1, 2, 3))
```

Here, the second add() method replaces the first one.

How to Achieve Method Overloading in Python

We can use default arguments.

```
class Test:
    def add(self, a, b, c=0):
        return a + b + c

obj = Test()

print(obj.add(1, 2))      # 2 parameters
print(obj.add(1, 2, 3))   # 3 parameters
```

We can also use variable-length arguments (*args).

```
class Test:
    def add(self, *args):
        return sum(args)

obj = Test()

print(obj.add(1, 2))
print(obj.add(1, 2, 3, 4))
```

Simple idea:
Python does not support real method overloading, but we can achieve similar behavior using default values or *args.

Q2) Method overriding: redefining a parent class method in the child class.

Method overriding means redefining a parent class method in the child class with the same name and parameters.

The child class provides its own implementation of the method.

Example:

```
class Parent:
    def show(self):
        print("This is Parent class method")


class Child(Parent):
    def show(self):   # overriding parent method
        print("This is Child class method")


obj = Child()
obj.show()
```

Output:

```
This is Child class method
```

Here, the show() method in the child class overrides the parent class method.

If you want to call the parent method inside the child class, use super():

```
class Child(Parent):
    def show(self):
        super().show()
        print("This is Child class method")
```

Simple idea:
Method overriding allows a child class to change the behavior of a parent class method.